



Quantum Walks and its Applications in Quantum Networks



Undergraduate Thesis - Andre Nogueira Ribeiro
Supervisor: Prof. Fabio Kon (IME/USP)
Co-supervisor: Prof. Antonio Jorge Gomes Abelém (UFPA)
Supervisor at University of Massachusetts: Prof. Don Towsley(UMass)

Contents

1	Abstract	2
2	Important Concepts	2
2.1	Linear Algebra	2
2.2	Quantum Computation and Quantum Information	3
3	Introduction to Quantum Computing	4
4	Quantum Algorithms	6
4.1	Deutsch's algorithm	7
4.2	Deutsch-Jozsa algorithm	8
4.3	Simon's algorithm	9
4.4	Grover's algorithm	11
4.5	Shor's algorithm	14
5	Introduction to Quantum Information	15
6	Introduction to Quantum Walks	15
7	Distributed Quantum Walk Control Plane for Quantum Networks	15
8	Future Possibilities	15

1 Abstract

Quantum computing emerged from the idea of building a machine capable of simulating complex quantum systems, a concept proposed by physicist Richard Feynman and mathematician Yuri Manin. The challenge with simulating quantum systems on classical computers is that the computational effort increases exponentially with the size of the system, making it impractical for any significantly large problems. In addition to this focus, research in quantum algorithms has also achieved notable success, with algorithms with better complexity than classical ones being proposed—such as Shor’s algorithm for number factorization.

Quantum computing is also an exciting field of study, offering a wide range of unexplored possibilities, which makes it both intriguing and promising. Currently, companies like Google and IBM, have developed functional quantum machines that highlight the importance of this technology. These machines are capable of performing computations that are inaccessible to classical computers due to their distinct physical properties. Some of the main challenges in advancing quantum computers include increasing the number of qubits (the basic unit of memory in quantum computing), reducing noise, and extending coherence time (the duration of quantum phenomena needed to execute an algorithm).

2 Important Concepts

This section describes some important concepts which will be needed to understand the content of this undergraduate thesis. You may read the concepts as they appear on this document.

2.1 Linear Algebra

- Hilbert space: A Hilbert space is a mathematical concept that extends the idea of Euclidean space to infinite dimensions. It is an inner product space, meaning it comes equipped with an inner product, which allows for the definition of norms. The space is also complete, meaning that any Cauchy sequence of vectors within the space converges to a point within the space. This property of completeness ensures that Hilbert spaces are well-suited for analysis, especially in quantum mechanics, where they provide the framework for representing quantum states and operators. The inner product in a Hilbert space allows for the measurement of orthogonality, projection, and distance, making it a foundational tool in functional analysis, quantum theory, and many other areas of mathematics and physics.
- $\langle \psi | \phi \rangle$: Inner product common representation.
- $|\psi\rangle |\phi\rangle$: Tensor product.
- Normal Operator: An operator A on V is normal if $A^\dagger A = AA^\dagger$. [2]
- Unitary evolution: An operator U is unitary if $U^\dagger U = UU^\dagger = I$. Unitary operators are normal. They are diagonalizable with respect to an orthonormal basis. Eigenvectors of a unitary operator associated with different eigenvalues are orthogonal. The eigenvalues have unit modulus, that is, they have the form $e^{i\alpha}$, where α is a real number. Unitary operators preserve the inner product, that is, the inner product of $U|v1\rangle$ and $U|v2\rangle$ is equal to the inner product of $|v1\rangle$ and $|v2\rangle$. The action of a unitary operator on a vector preserves its norm [2]. A complex square matrix U is called unitary if the columns of U form an orthonormal set.
- Hermitian Operator: An operator A on V is Hermitian or self-adjoint if $A^\dagger = A$. [2]
- Positive Operator: A positive operator A is defined to be an operator such that for any vector $|v\rangle, \langle v|$, $A|v\rangle$ is a real, non-negative number. If $\langle v|, A|v\rangle$ is strictly greater than zero for all $|v\rangle \neq 0$ then we say that A is positive definite. In Exercise 2.24 on this page you will show that any positive operator is automatically Hermitian, and therefore by the spectral decomposition has diagonal representation $\sum_i \lambda_i |i\rangle \langle i|$, with non-negative eigenvalues λ_i . [1]
- Wavefunction: On the quantum computing field, a wavefunction is a function that the x axis corresponds to the basis states and the y axis corresponds to the quantum amplitude of the state.

- **Orthogonal Projector:** An operator P on V is an orthogonal projector if $P^2 = P$ and $P^\dagger = P$. [2]
- **With high probability (WHP):** In mathematics, an event that occurs with high probability is one whose probability depends on a certain number n and goes to 1 as n goes to infinity.
- **Matrix rank:** In linear algebra, the rank of a matrix A is the dimension of the vector space generated (or spanned) by its columns. This corresponds to the maximal number of linearly independent columns of A .

2.2 Quantum Computation and Quantum Information

- **Superposition:** In quantum mechanics, superposition refers to the ability of a quantum system to exist in multiple states simultaneously. A quantum particle, such as an electron, can be in a combination of different states until it is measured, at which point it collapses into one definite state. Superposition is a non intuitive fundamental principle that enables the unique behavior of quantum systems.
- **Entanglement:** In quantum mechanics, entanglement is a phenomenon where two or more particles become linked in such a way that the state of one particle instantly influences the state of the other(s), no matter how far apart they are. This correlation persists even when the particles are separated by large distances. Entanglement is a key feature of quantum mechanics, enabling phenomena such as quantum teleportation and playing a crucial role in quantum computing and cryptography.
- **Phase (relative):** In quantum computing, phase refers to the angle that describes the position of a qubit's state in relation to a reference point. When a qubit is in superposition (a combination of both 0 and 1), its state has a magnitude (how much it is in 0 or 1) and a phase (which affects how the states interact). Phases are important because they influence the interference between quantum states, which can change the probability of measuring certain outcomes. The relative phase between states is critical for how quantum algorithms work and how qubits interact.
- **Global phase:** In quantum computing, the global phase refers to an overall phase factor applied to a quantum state that does not affect measurement outcomes. A quantum state is typically represented as a vector, and multiplying the entire state by a complex number of unit magnitude (e.g., $e^{i\theta}$) introduces a global phase. This phase has no observable consequences, as quantum measurements are only sensitive to relative phases between components of a superposition. Therefore, global phase usually can be ignored in practical computations and is considered irrelevant to the behavior or outcomes of a quantum system.
- **Quantum Error Correction:** Quantum error correction is a set of techniques designed to protect quantum information from errors due to decoherence, noise, or other quantum disturbances. In quantum systems, qubits are fragile and susceptible to errors from their environment, which can lead to loss of information. Quantum error correction encodes logical qubits into entangled states of multiple physical qubits, allowing for the detection and correction of errors without directly measuring and collapsing the quantum state. It is a critical component in making quantum computers reliable and scalable for long computations, ensuring the integrity of quantum information over time.
- **Pure or mixed state:** On a Bloch sphere, pure states are represented by a point on the surface of the sphere, whereas mixed states are represented by an interior point. The completely mixed state of a single qubit is represented by the center of the sphere, by symmetry. The purity of a state can be visualized as the degree in which it is close to the surface of the sphere. In quantum mechanics, the state of a quantum system is represented by a state vector (or ket) $|\Psi\rangle$. A quantum system with a state vector $|\Psi\rangle$ is called a pure state. However, it is also possible for a system to be in a statistical ensemble of different state vectors: For example, there may be a 50% probability that the state vector is $|\Psi_1\rangle$ and a 50% chance that the state vector is $|\Psi_2\rangle$. This system would be in a mixed state. The density matrix is especially useful for mixed states, because any state, pure or mixed, can be characterized by a single density matrix.

- **Measurement:** Quantum measurements are described by a collection M_m of measurement operators. These are operators acting on the state space of the system being measured. The index m refers to the measurement outcomes that may occur in the experiment. If the state of the quantum system is $|\psi\rangle$ immediately before the measurement then the probability that result m occurs is given by $p(m) = \langle\psi| M_m^\dagger M_m |\psi\rangle$ and the state of the system after the measurement is $\frac{M_m|\psi\rangle}{\sqrt{\langle\psi| M_m^\dagger M_m |\psi\rangle}}$. The measurement operators satisfy the completeness equation, $\sum_m M_m^\dagger M_m = I$. The completeness equation expresses the fact that probabilities sum to one: $1 = \sum_m p(m) = \sum_m \langle\psi| M_m^\dagger M_m |\psi\rangle$. [1]
- **Projective measurement:** A projective measurement is described by an observable, M , a Hermitian operator on the state space of the system being observed. The observable has a spectral decomposition, $\sum_m m P_m$, where P_m is the projector onto the eigenspace of M with eigenvalue m . The possible outcomes of the measurement correspond to the eigenvalues, m , of the observable. Upon measuring the state $|\psi\rangle$, the probability of getting result m is given by $p(m) = \langle\psi| P_m |\psi\rangle$. Given that outcome m occurred, the state of the quantum system immediately after the measurement is $\frac{P_m|\psi\rangle}{\sqrt{p(m)}}$. The average value of the observable M is often written $\langle M \rangle = \langle\psi| M |\psi\rangle$. [1]
- **Fidelity:** A second measure of distance between quantum states is the fidelity. The fidelity is not a metric on density operators, but we will see that it does give rise to a useful metric. This section reviews the definition and basic properties of the fidelity. The fidelity of states ρ and σ is defined to be $F(\rho, \sigma) = \text{tr} \sqrt{\rho^{\frac{1}{2}} \sigma \rho^{\frac{1}{2}}}$. Fidelity does satisfy many of the same properties as the trace distance. For example, it is invariant under unitary transformations: $F(U\rho U^\dagger, U\sigma U^\dagger) = F(\rho, \sigma)$. The fidelity between a pure state $|\psi\rangle$ and an arbitrary state ρ is $\langle\psi| \rho |\psi\rangle$. [1]
- **Quantum circuit depth:** The depth of a circuit is a metric that calculates the longest path between the data input and the output. Each gate counts as a unit.
- **Partial trace:** It can be calculated as the $\sum_B \langle\phi_B| \rho_A \otimes \rho_B |\phi_B\rangle = \rho_A \otimes \sum_B \langle\phi_B| \rho_B |\phi_B\rangle$. In a more clarified way $\text{tr}_A(L_{AB}) = \sum_i \langle i|_A L_{AB} |i\rangle_A = \langle 0|_0 \langle 0|_0 (\langle 1|_1 \langle 0|_1)_B + \langle 1|_0 \langle 1|_1 (\langle 0|_0 \langle 0|_0)_B = (\langle 1|_1 \langle 0|_1)_B$.

3 Introduction to Quantum Computing

Qubit A qubit (quantum bit) is the fundamental unit of memory in a quantum computer, and unlike classical bits, which are limited to being in a state of either 0 or 1, a qubit can exist in a superposition of both 0 and 1 simultaneously. This unique property is a consequence of quantum mechanics, which allows qubits to represent not just one of two discrete states, but an infinite number of possibilities between 0 and 1.

In more technical terms, a qubit's state is described as a quantum superposition, where it can be in a combination of both 0 and 1, weighted by complex numbers called probability amplitudes. However, this superposition exists only until the qubit is measured. Upon measurement, the qubit collapses to a definite state, either 0 or 1, with probabilities determined by its probability amplitudes before the measurement.

The power of quantum computing lies in the ability to leverage these superposition states without directly measuring them. Through quantum algorithms, it's possible to manipulate and process data while qubits remain in superposition, exploring multiple possible solutions simultaneously. This parallelism can provide a significant computational advantage over classical computers, in problems such as factoring large numbers or simulating quantum systems.

Moreover, qubits also exhibit another quantum property called entanglement, where the state of one qubit is intrinsically linked to the state of another, regardless of the distance between them. This further enhances the computational potential of quantum systems, enabling qubits to work together in ways that classical bits cannot, exponentially increasing the power of quantum computations.

Mathematical Model. Studying quantum computing requires a understanding of linear algebra, as it forms the mathematical foundation for describing quantum systems. I will briefly cover the key concepts. Every quantum system can be represented as a vector in a Hilbert space. A qubit, the fundamental unit of information in a quantum computer, can be represented as a two-dimensional vector, and the operations, or quantum gates, that act on this qubit are represented by unitary matrices. Unitary matrices preserve the norm of the vector, ensuring the system's total probability remains 1. The state of a qubit is expressed as:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

where $|0\rangle$ and $|1\rangle$ are basis vectors, defined as $(1, 0)$ and $(0, 1)$ respectively. These basis states correspond to the classical bit values 0 and 1. The complex numbers a and b are the probability amplitudes. The squared magnitudes, $|a|^2$ and $|b|^2$, represent the probabilities of the qubit collapsing to the state 0 or 1, respectively, upon measurement. For these probabilities to make physical sense, the normalization condition must be satisfied:

$$|a|^2 + |b|^2 = 1$$

This ensures that the sum of probabilities for all possible outcomes equals 1.

When we move beyond a single qubit, the dimensionality of the system increases exponentially. A system of two qubits is represented by a four-dimensional vector, and in general, a system of n qubits is described by a vector in a 2^n -dimensional space. This exponential growth in dimensionality is one thing which gives quantum computers advantage when compared to classical computers.

A common and intuitive way to visualize the state of a single qubit is through the Bloch sphere. The Bloch sphere represents the qubit as a point on the surface of a three-dimensional sphere, where the angles of the point correspond to the probability amplitudes and the global phase of the qubit. The global phase, while often not directly measurable, plays a crucial role in the evolution of the quantum system.

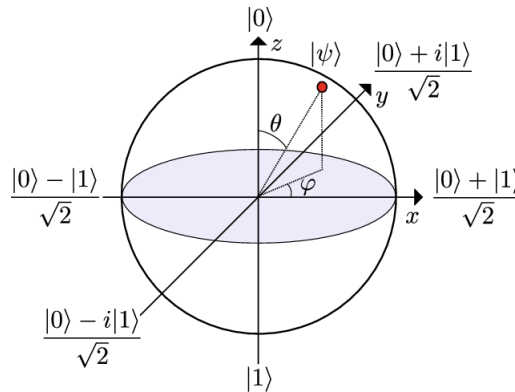


Figure 1: Bloch Sphere - <https://www.researchgate.net/figure/The-Bloch-sphere-representation-of-a-qubit-state-The-north-pole-is-the-ground-state-fig1335845744>

In addition to these representations, understanding quantum systems requires grasping the probabilistic nature of quantum mechanics. The outcomes of quantum measurements are not deterministic but are governed by probability distributions, dictated by the amplitudes of the quantum state vector. This probabilistic behavior, though counterintuitive from a classical perspective, is a key aspect of quantum computing and forms the basis for algorithms that can outperform their classical counterparts.

Quantum Gates In a quantum circuit, gates play a crucial role in manipulating the quantum information that passes through them. Unlike classical gates, which operate on bits, quantum gates operate on qubits and are represented mathematically by unitary matrices. These matrices are square, meaning that a gate acting on a single qubit is represented by a 2×2 matrix, while a gate acting on two qubits is represented by a 4×4 matrix. The unitary nature of these matrices ensures that the operations are reversible, a key property of quantum computations.

Quantum gates are the building blocks of quantum algorithms, and understanding how they function is essential for constructing and analyzing quantum circuits. Below are some important quantum gates commonly used in quantum computations:

The Hadamard gate creates a superposition of quantum states, transforming a qubit from a definite state (either $|0\rangle$ or $|1\rangle$) into an equal probability superposition of both states. This gate is crucial for many quantum algorithms, such as Grover's search algorithm and Shor's factoring algorithm. Its matrix representation is:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The Pauli-X gate, also known as the quantum NOT gate, flips the state of a qubit. If the qubit is in the state $|0\rangle$, the X gate transforms it to $|1\rangle$, and vice versa. Its matrix representation is:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

The CNOT (Controlled-Not) gate is a two-qubit gate that operates conditionally. Considering two qubits, control and target, it flips the state of the target qubit if and only if the control qubit is in the state $|1\rangle$. The CNOT gate plays a fundamental role in quantum entanglement and error correction. Its matrix representation for two qubits is:

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

The Toffoli gate is a three-qubit gate, also known as the controlled-controlled-X gate. It flips the state of the third qubit (the target) if and only if the first two qubits (the control qubits) are both in the state $|1\rangle$. This gate is essential for implementing reversible classical logic and can be used for quantum error correction.

$$CCX = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

The phase gate adds a phase factor to the quantum state without changing the probabilities of measuring the qubit in $|0\rangle$ or $|1\rangle$. Its matrix representation is:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

The T gate is a phase gate that applies a phase shift of $\frac{\pi}{4}$ to the state $|1\rangle$. It is an important component for creating universal quantum gates and can be represented as:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix}$$

!!!!adicionar exemplos de circuitos e operacoes mais simples indicando a notacao utilizada

4 Quantum Algorithms

Quantum algorithms are computational processes designed to harness the unique properties of quantum mechanics to solve certain problems more efficiently than classical algorithms.

One of the primary goals in developing quantum algorithms is to identify processes that can be executed with a polynomial number of quantum gates. In quantum computing, gates are the operations

that manipulate qubits and thus form the steps of an algorithm. Algorithms requiring a polynomial number of gates are desirable because they remain efficient as problem sizes grow, ensuring that the quantum resources needed stay manageable.

By focusing on algorithms that require polynomial resources, researchers aim to make practical quantum computing a reality, enabling breakthroughs in fields such as cryptography, optimization, and materials science. In this section I will present some fundamentals and important quantum algorithms. Some of this content was based on learning provided by the IBM quantum material !!!! and qiskit.org

4.1 Deutsch's algorithm

Deutsch's algorithm is one of the earliest and simplest quantum algorithms, showcasing how quantum computing can solve certain types of problems more efficiently than classical computing. The problem it addresses is a simple example that demonstrates the power of quantum parallelism.

Imagine a black-box function, $f(x)$, that takes a single binary input x (either 0 or 1) and returns either 0 or 1. So, $f(x)$ has two possible inputs (0 and 1) and two possible outputs (0 or 1). There are four possible types of functions $f(x)$:

1. $f(0) = 0$ and $f(1) = 0$ (constant function returning 0 for both inputs)
2. $f(0) = 1$ and $f(1) = 1$ (constant function returning 1 for both inputs)
3. $f(0) = 0$ and $f(1) = 1$ (balanced function returning different values for each input)
4. $f(0) = 1$ and $f(1) = 0$ (also a balanced function)

The goal is to determine whether $f(x)$ is constant (both outputs are the same) or balanced (outputs are different).

Classically, we would need to check both inputs of $f(x)$ (i.e., $f(0)$ and $f(1)$) to determine if it is constant or balanced. This requires two function evaluations.

Quantumly, however, Deutsch's algorithm allows us to determine if $f(x)$ is constant or balanced with just one evaluation of $f(x)$. This efficiency comes from the ability of a qubit to exist in a superposition, allowing us to evaluate both inputs simultaneously. Below is how it works.

1. We start with two qubits:
 - The first qubit, in state $|0\rangle$, will hold the input to the function $f(x)$.
 - The second qubit, in state $|1\rangle$, is used to store the result of the function evaluation.

So, our initial state is:

$$|\psi_0\rangle = |0\rangle \otimes |1\rangle = |0, 1\rangle$$

2. Next, we apply the Hadamard gate to both qubits. The Hadamard gate creates a superposition state by transforming:

- $|0\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$
- $|1\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$

After applying Hadamard gates to both qubits, our state becomes:

$$|\psi_1\rangle = \frac{1}{2}(|0\rangle + |1\rangle) \otimes (|0\rangle - |1\rangle)$$

This sets up the first qubit in a superposition of inputs 0 and 1, allowing us to evaluate both cases simultaneously.

3. Now, we apply the "oracle" or function gate, which encodes $f(x)$ into the second qubit. This operation, called the quantum oracle, flips the second qubit's sign based on the value of $f(x)$ without collapsing the state:

$$|\psi_2\rangle = \frac{1}{2}(|0\rangle \otimes (|0 \oplus f(0)\rangle - |1 \oplus f(0)\rangle) + |1\rangle \otimes (|0 \oplus f(1)\rangle - |1 \oplus f(1)\rangle))$$

- Next, we apply a Hadamard gate to the first qubit again. This final Hadamard gate transforms the state in such a way that, if $f(x)$ is constant, the first qubit will return to $|0\rangle$; if $f(x)$ is balanced, it will end up in $|1\rangle$.

After applying this Hadamard gate, we measure the first qubit:

- If the measurement outcome is 0, $f(x)$ is constant.
- If the measurement outcome is 1, $f(x)$ is balanced.

The magic of Deutsch's algorithm lies in the ability of the quantum oracle and superposition to use interference (through the Hadamard gates) to reveal whether $f(x)$ is constant or balanced with just one evaluation. This is impossible in classical computing, where we would need at least two evaluations to determine the nature of $f(x)$.

Below is the quantum circuit associated with Deutsch's algorithm.

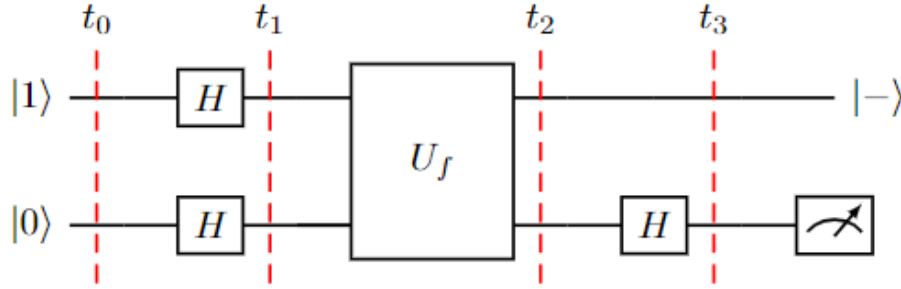


Figure 2: https://www.researchgate.net/figure/Graphical-representation-of-the-quantum-circuit-for-the-Deutsch-algorithm_fig5381574807

4.2 Deutsch-Jozsa algorithm

The Deutsch-Jozsa algorithm is an extension of Deutsch's algorithm. It deals with a specific type of function. Suppose we have a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, meaning it takes an n -bit binary string as input and outputs either 0 or 1. The function $f(x)$, again, can be of two types: constant or balanced. In this case balanced means that the function outputs 0 for half of the inputs and 1 for the other half. Constant has the same meaning as before.

The goal is to determine if $f(x)$ is constant or balanced. Classically, we would need to evaluate $f(x)$ up to $2^{n-1} + 1$ times in the worst case to be certain, because we'd need to check enough inputs to ensure that it's not alternating between 0s and 1s.

In contrast, the Deutsch-Jozsa algorithm can determine whether $f(x)$ is constant or balanced with just one evaluation of the function, demonstrating an exponential speedup over classical approaches. Below is how it works

- We start with $n + 1$ qubits. The first n qubits represent the input to the function $f(x)$, and the last qubit will help with the evaluation:

- Set the first n qubits to $|0\rangle$.
- Set the last qubit to $|1\rangle$.

So our initial state is:

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |1\rangle$$

- Apply the Hadamard gate to each of the $n + 1$ qubits. The Hadamard gate puts each qubit into a superposition, so we end up with all possible inputs (for the first n qubits) in superposition simultaneously. This transforms the state to:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

The first n qubits are now in a superposition of all possible inputs, while the last qubit is in a superposition that allows it to act as a phase kickback in the next step.

3. Apply the Oracle (Quantum Function): Now we use a quantum oracle that applies the function $f(x)$ to the superposition state. The oracle is a black-box function that, for each input x , flips the phase of the state if $f(x) = 1$, but does nothing if $f(x) = 0$. This modifies our state to:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

Here, $(-1)^{f(x)}$ introduces a phase factor based on the value of $f(x)$, changing the amplitude of each input state in a way that allows us to determine whether $f(x)$ is constant or balanced.

4. Apply the Hadamard gate again to the first n qubits, transforming our state once more. This step is crucial, as it amplifies the amplitudes in a way that depends on whether $f(x)$ is constant or balanced. After this transformation, the state will collapse into one of two distinct results based on the type of $f(x)$.
5. Finally, we measure the first n qubits:
 - If the measurement outcome is $|0\rangle^{\otimes n}$, $f(x)$ is constant.
 - If the measurement outcome is anything other than $|0\rangle^{\otimes n}$, $f(x)$ is balanced.

The Deutsch-Jozsa algorithm exploits quantum parallelism and interference. By putting the input qubits in a superposition, we can evaluate $f(x)$ for all possible inputs at once. The phase kickback and interference through the Hadamard gates allow us to detect whether the function outputs are the same (constant) or balanced across the inputs, without needing multiple evaluations.

Below is the quantum circuit associated with Deutsch-Jozsa algorithm.

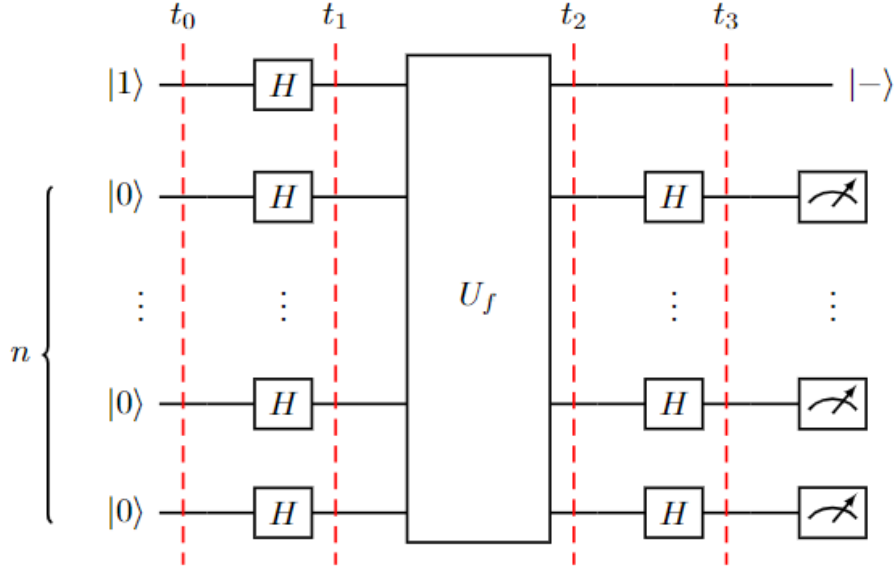


Figure 3: https://www.researchgate.net/figure/Graphical-representation-of-the-quantum-circuit-for-the-Deutsch-Jozsa-algorithm_fig6381574807

4.3 Simon's algorithm

Simon's algorithm is one of the foundational quantum algorithms, demonstrating an exponential speedup over classical algorithms for a specific type of problem. It was a precursor to Shor's algorithm

and is significant because it proved that quantum computing could solve certain problems exponentially faster than classical computing.

In Simon's problem, we are given a black-box function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with the promise that there exists a secret string $s \in \{0, 1\}^n$ such that:

- If $f(x) = f(y)$, then $x \oplus y = s$.

This means that $f(x)$ is 2-to-1: each output is produced by exactly two distinct inputs, and the difference (XOR) between these two inputs is the secret string s . The goal of Simon's algorithm is to determine the value of s .

Classically, solving Simon's problem requires $O(2^{n/2})$ evaluations of $f(x)$ to find s with high probability. In contrast, Simon's algorithm can find s using only $O(n)$ queries to $f(x)$ by taking advantage of quantum parallelism and interference.

1. Start with $2n$ qubits, where the first n qubits represent the input x and the second n qubits will store the output of $f(x)$. Initialize all qubits to $|0\rangle$:

$$|\psi_0\rangle = |0\rangle^{\otimes 2n}$$

2. Apply Hadamard gates to the first n qubits, putting them into a superposition of all possible inputs. The state becomes:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |0\rangle^{\otimes n}$$

3. Next, apply the oracle for $f(x)$, which maps $|x\rangle \otimes |0\rangle$ to $|x\rangle \otimes |f(x)\rangle$. After the oracle query, the state becomes:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |f(x)\rangle$$

Since $f(x)$ is 2-to-1, pairs of inputs differ by the secret string s and map to the same output. The result is an entangled state where knowing one x value gives a relationship to the other.

4. Measure the second n qubits. This collapses the state, and we are left with a superposition of the first register that includes only those x -values satisfying $x \oplus y = s$. This reduces our state to:

$$|\psi_3\rangle = \frac{1}{\sqrt{2}} (|x\rangle + |x \oplus s\rangle)$$

for some random x .

5. Apply Hadamard gates to the first n qubits. This transforms each $|x\rangle$ and $|x \oplus s\rangle$ component, giving us a new state where we can measure information about s :

$$|\psi_4\rangle = \frac{1}{\sqrt{2^n}} \sum_{z=0}^{2^n-1} (-1)^{x \cdot z} (|z\rangle + (-1)^{s \cdot z} |z\rangle)$$

The measurement yields a bitstring z that satisfies $z \cdot s = 0 \pmod{2}$, giving us information about the secret string s .

6. Repeat the process n times to get n linearly independent equations for s . Solve this system of linear equations to determine s .

This provides an exponential speedup, requiring only $O(n)$ evaluations instead of $O(2^{n/2})$.

Simon's algorithm was one of the first to demonstrate an exponential quantum speedup. It shows how quantum algorithms can solve specific types of problems faster than classical algorithms by leveraging superposition and interference. The principles used in Simon's algorithm laid the groundwork for more complex algorithms, such as Shor's algorithm for factoring.

Below is the quantum circuit associated with Simon's algorithm.

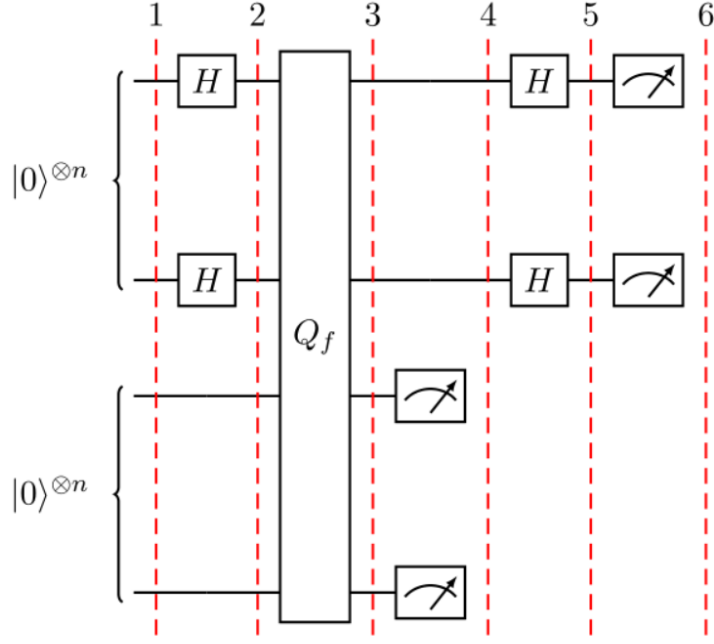


Figure 4: <https://medium.com/@monitsharma/learn-quantum-computing-with-qiskit-simons-algorithm-8525adecd091>

4.4 Grover's algorithm

Grover's algorithm is a quantum search algorithm that provides a quadratic speedup over classical search algorithms. It is used to search an unsorted database or solve search problems, where the goal is to locate a specific "marked" item among N possible entries. While a classical search requires $O(N)$ operations to find the item in the worst case, Grover's algorithm can find it in $O(\sqrt{N})$ operations.

The problem assumes a black-box function (oracle) $f(x)$ defined as:

$$f(x) = \begin{cases} 1 & \text{if } x \text{ is the marked item,} \\ 0 & \text{otherwise.} \end{cases}$$

The oracle function identifies the marked item by returning 1 for the correct input $x = x_0$ and 0 otherwise. The goal is to find x_0 using as few queries to the oracle as possible.

Grover's algorithm is optimal for problems where the marked item needs to be located within an unstructured search space, making it useful for various applications in cryptography, optimization, and more.

1. Begin with n qubits in the $|0\rangle$ state. Apply a Hadamard gate to each qubit to create an equal superposition of all possible states:

$$|\psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

where $N = 2^n$, the total number of possible states for n qubits.

2. Apply the oracle to flip the phase of the marked item. The oracle changes the amplitude of the target state $|x_0\rangle$ by introducing a phase of -1. The state after the oracle application becomes:

$$|\psi_1\rangle = \frac{1}{\sqrt{N}} \sum_{x \neq x_0} |x\rangle - \frac{1}{\sqrt{N}} |x_0\rangle$$

This marks the target item by shifting its amplitude, setting it up for amplification in the next steps.

3. Apply the Grover diffusion operator, which increases the amplitude of the marked state while decreasing the amplitudes of the other states. This operator consists of three steps:
 - Apply a Hadamard transform to all qubits.
 - Perform a phase flip on the $|0\rangle$ state (invert the amplitudes about the average).
 - Apply the Hadamard transform to all qubits again.

After the amplitude amplification step, the state evolves closer to the target state. The diffusion operator is represented by:

$$D = 2|\psi_0\rangle\langle\psi_0| - I$$

4. Repeat the oracle and diffusion steps approximately $O(\sqrt{N})$ times. Each iteration increases the amplitude of the marked state and decreases the amplitudes of the unmarked states, converging on the solution.
5. After performing the oracle and amplitude amplification steps $O(\sqrt{N})$ times, measure the qubits. With high probability, the result will be the marked item x_0 .

The oracle marks the target state by inverting its phase, and the diffusion operator amplifies the amplitude of the marked state by rotating it closer to the target. By iterating these steps $O(\sqrt{N})$ times, Grover's algorithm amplifies the target state's amplitude, making it significantly more likely to be observed upon measurement.

Grover's algorithm illustrates the power of quantum computing for search problems. By reducing the search complexity from $O(N)$ to $O(\sqrt{N})$, it provides a quadratic speedup over classical methods. Although it is not an exponential improvement, this quadratic speedup is highly valuable for large search spaces and has potential applications in cryptography, database searching, and optimization.

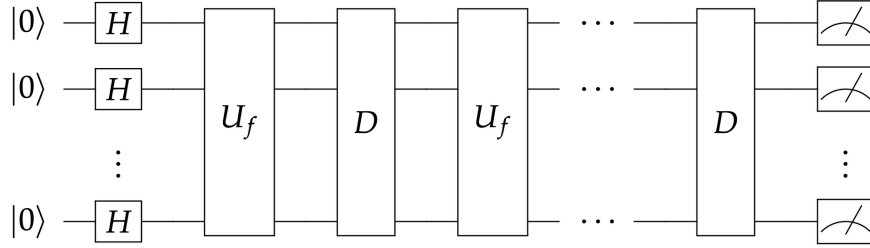


Figure 5: <https://towardsdatascience.com/grovers-quantum-search-algorithm-54c427315768>

For an easier understanding, Grover's algorithm has a geometric interpretation, which provides insight into why it efficiently locates the marked item. The algorithm's core process—phase inversion and amplitude amplification—can be visualized as a sequence of rotations in a multi-dimensional space.

Imagine that the quantum state space is represented by a vector space where each possible basis state (each possible value of x in the search space) corresponds to a unique axis in the space. In this space, we can define two important vectors:

- $(|x_0\rangle)$: This vector represents the quantum state corresponding to the marked item, which we want to identify.
- $(|s\rangle)$: This vector represents an equal probability amplitude over all possible states, constructed as:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

where $N = 2^n$, the total number of possible states for n qubits.

The initial state of the algorithm, after applying Hadamard gates to all qubits, is $|s\rangle$, a uniform superposition over all possible items.

We can think of $|s\rangle$ as forming an angle with $|x_0\rangle$ (the solution state). Since $|s\rangle$ is an equal superposition of all possible states, it can be decomposed into two components:

- A component along the direction of $|x_0\rangle$.
- A component perpendicular to $|x_0\rangle$ (we can call this the "non-solution" space).

Let $|w\rangle$ represent the direction orthogonal to $|x_0\rangle$, which spans all states except the marked state. Thus, we can decompose $|s\rangle$ as:

$$|s\rangle = \alpha |x_0\rangle + \beta |w\rangle$$

where α and β are real coefficients representing the projection of $|s\rangle$ onto $|x_0\rangle$ and $|w\rangle$, respectively.

Grover's algorithm uses two main operations that act as rotations in this vector space:

1. The oracle marks the solution state by flipping the phase of $|x_0\rangle$, effectively rotating $|s\rangle$ through the angle between $|s\rangle$ and $|x_0\rangle$. This is equivalent to reflecting $|s\rangle$ across the hyperplane orthogonal to $|x_0\rangle$.
2. The Grover diffusion operator acts as a reflection about $|s\rangle$. This reflection moves the vector closer to $|x_0\rangle$ by effectively doubling the amplitude of the marked state (the component of $|s\rangle$ in the $|x_0\rangle$ direction).

Each application of the oracle and diffusion operators constitutes one "Grover iteration." These two reflections, applied in sequence, effectively rotate the state vector by a fixed angle closer to $|x_0\rangle$ in each iteration.

The rotation angle per Grover iteration is approximately:

$$\theta = 2 \arcsin\left(\frac{1}{\sqrt{N}}\right)$$

This angle depends on the size of the search space, N , and determines how quickly the algorithm converges to the solution state. To maximize the probability of measuring the marked state $|x_0\rangle$, Grover's algorithm requires approximately:

$$k = \frac{\pi}{4} \sqrt{N}$$

iterations, or applications of the oracle and diffusion operators, to rotate the state vector as close as possible to $|x_0\rangle$. Below is an exemplification image.

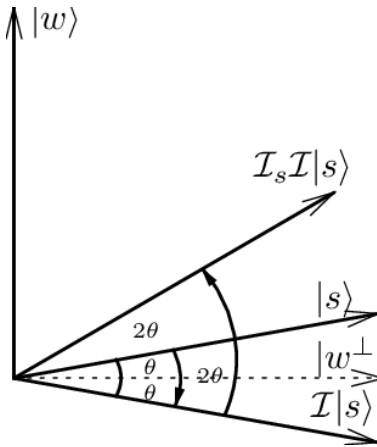


Figure 6: https://www.researchgate.net/figure/Geometric-visualization-of-Grovers-algorithm_fig147438604

After $O(\sqrt{N})$ iterations, the state vector will be very close to $|x_0\rangle$, and a measurement of the state will yield the marked item with high probability. This geometric interpretation highlights why Grover's algorithm is optimal for unstructured search: each iteration moves the state vector closer to the target solution, and the quadratic speedup comes from the number of rotations required to reach the solution state, which scales as $\sqrt{N}$.

!!! colocar uma imagenzinha da interpretacao geometrica

4.5 Shor's algorithm

Shor's algorithm is a quantum algorithm that efficiently finds the prime factors of a large integer N , solving the integer factorization problem. This problem is crucial because the security of many classical cryptographic systems, such as RSA, relies on the difficulty of factoring large integers. Shor's algorithm achieves exponential speedup over the best-known classical algorithms, making it a key example of quantum computing's potential.

Given a large integer N , Shor's algorithm aims to find a nontrivial factor of N . The algorithm relies on the following insight from number theory:

- For a random integer a (where $1 < a < N$), if a and N are coprime!!!, then there exists a smallest positive integer r (called the order) such that:

$$a^r \equiv 1 \pmod{N}$$

- If r is even, then with high probability, the factors of N are given by the greatest common divisor of N with $a^{r/2} - 1$ and $a^{r/2} + 1$.

Thus, finding the period r allows us to factor N . Classically, finding r would require exponential time, but quantum computing can efficiently determine r using a technique called the Quantum Fourier Transform (QFT). Following are the algorithm steps.

1. Choose a random integer a where $1 < a < N$ and $\gcd(a, N) = 1$.
2. The main quantum component of Shor's algorithm is finding the period r of the function $f(x) = a^x \pmod{N}$. To find r , we set up a quantum state that represents this function.
3. Initialize two quantum registers:
 - The first register holds an equal superposition of all possible values of x (from 0 to some $2^n - 1$), created using Hadamard gates.
 - The second register starts in the $|0\rangle$ state and will store the values $f(x) = a^x \pmod{N}$.

The initial state after applying Hadamard gates to the first register is:

$$|\psi_0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |0\rangle$$

4. Using a quantum circuit, apply the function $f(x) = a^x \pmod{N}$ to entangle the first register with the output in the second register:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |f(x)\rangle$$

This state encodes periodicity information related to the order r .

5. To extract the period r , we apply the Quantum Fourier Transform to the first register. The QFT transforms a superposition of states into a frequency space that reveals periodic patterns.

Quantum Fourier Transform (QFT)

The Quantum Fourier Transform is the quantum analog of the discrete Fourier transform (DFT) and is essential for finding periodicity in quantum algorithms. For a state $|x\rangle = \sum_{k=0}^{N-1} x_k |k\rangle$, the QFT is defined as:

$$\text{QFT}(|x\rangle) = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i xy/N} |y\rangle$$

where $N = 2^n$ for an n -qubit system.

In Shor's algorithm, the QFT allows us to move from a superposition over the computational basis to one where the periodicity of $f(x) = a^x \bmod N$ becomes measurable.

The QFT circuit consists of Hadamard gates and controlled phase rotations, with the following steps for an n -qubit register:

- (a) Apply a Hadamard gate to the first qubit.
- (b) Apply controlled phase rotations between each qubit, such that each subsequent qubit has a phase shift based on the values of the previous qubits.
- (c) Repeat this process for each qubit, applying decreasing rotations as the circuit progresses.
- (d) Finally, perform a "swap" operation to reverse the order of the qubits.

The QFT transforms the state so that measuring it will reveal information about the period r in the frequency domain.

6. After applying the QFT, measure the first register. The result will likely yield an integer k that is related to the period r by:

$$k \approx \frac{j}{r}$$

for some integer j . Using continued fractions, we can compute r from k with high probability.

7. Once r is found, check if it is even. Then compute the factors of N as $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$. If these values yield nontrivial factors, the algorithm is complete. Otherwise, repeat the process with a new random a .

Shor's algorithm leverages the quantum superposition and the Quantum Fourier Transform to efficiently find the period r , a task that is exponentially hard on classical computers. By encoding the periodicity of $f(x) = a^x \bmod N$ in a quantum state and using the QFT to extract this periodicity, Shor's algorithm allows us to find factors of N with only polynomial quantum resources.

Shor's algorithm demonstrates the power of quantum computing for number-theoretic problems, achieving an exponential speedup in factoring large numbers. The algorithm combines classical number theory, modular exponentiation, and the Quantum Fourier Transform, which plays a central role in revealing periodicity. This algorithm has substantial implications for cryptography, as it provides an efficient means of breaking widely-used cryptographic systems like RSA.

Below is the associated quantum circuit.

5 Introduction to Quantum Information

6 Introduction to Quantum Walks

7 Distributed Quantum Walk Control Plane for Quantum Networks

8 Future Possibilities

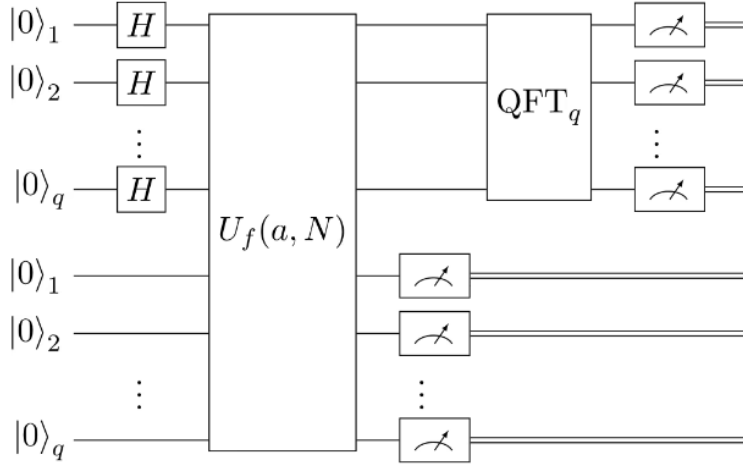


Figure 7: [https : //prefetch.eu/know/concept/shors – algorithm/](https://prefetch.eu/know/concept/shors-algorithm/)

References

- [1] C. I. Nielsen Michael A. *Quantum computation and quantum information: 10th Anniversary Edition*. New York: Cambridge University Press, 2010.
- [2] R Portugal. *Quantum Walks, and Search Algorithms*. Switzerland: Springer Nature, 2018.